How the System Checks Student and Scholarship Records

The Same Findings, Explained in Simple English

This guide answers the same six questions as the earlier version. The facts, percentages and warnings have not changed. Only the explanation is simpler.

Important: The answers come from the saved program code and database-update files. They do not mean that we tested every rule on the live website. The related thesis parts are on pages 21-22 and 40-42 of the original paper.

Word Guide / Legend

Use this list whenever an unfamiliar word appears in the next pages.

Word	Simple Meaning
Record / field	A record is saved information about one person or application. A field is one detail, such as a name or Student ID.
Duplicate / possible duplicate	Two records that may describe the same student. A possible match still needs careful checking.
Grantor / roster	A grantor provides a scholarship. A roster is an official list of scholars.
Exact match	The values are the same after the program's text-cleaning rules are applied.
Levenshtein	A way to count letter changes between two texts. A change can add, remove or replace a letter. For example, Ana becomes Anna by adding one n.
Fuzzy matching / similarity	Checking how close two texts are, even if they are not exactly the same. Levenshtein helps calculate this.
Weight / weighted score	A weight says how much a detail counts. The weighted score combines the comparisons. Student ID counts more than province.
Threshold	The minimum score needed for a rule to apply. At least 84% means 84% or higher.
Token / token overlap	A token is a word in a name. Token overlap measures shared name words. Token-sorted means putting the words in alphabetical order.
Scholarship commitment	The system has linked the student to a scholarship and prevents another conflicting scholarship commitment.
Conflict / flag / audit	A conflict means records or assignments disagree. A flag marks something for checking. An audit checks records for possible problems.
Backend / UI	The backend checks and saves information on the server. The UI is the screen and buttons the user sees.

1. Which Student Details Are Compared?

The weighted duplicate checker compares these details. The percentage shows how much each detail counts in the combined score when all comparisons are available.

Student Detail	How Much It Counts
Student ID	32%
Full name	30%
Email	10%
Contact number	8%
Course	8%
Year level	4%
City or municipality	4%
Province	4%

Birthdate and school are not used in this score. For the address, this checker uses only city or municipality and province, not the full address.

If a detail is missing, the checker leaves that comparison out and adjusts the total using the weights of the comparisons it can use. It also compares names in their original word order and with their words in alphabetical order. It keeps whichever name score is higher.

Where this is written in the code: backend/grantor_algorithms.py:68

2. Does It Always Check Student ID First?

No. There is no single checking order for every task. Different tasks have different checks:

- **Creating an account:** after checking the documents, the system looks for the Student ID in the scholar list and checks whether the names agree. It then checks whether an account with that ID already exists.

- **Adding a scholar or uploading a scholar list:** the server checks the scholarship, repeated rows, Student ID and name, existing active scholar-list assignments, and scholarship commitments.

- **Using the weighted duplicate checker:** it compares the records given to it. That checker does not first search the database for active scholarships.

For the paper: Do not say that every process follows 'check the exact Student ID against active scholars first, then use similar names if needed.' That does not describe every part of the current program.

Where this is written in the code: backend/signup_service.py:546; backend/roster_workflow_service.py:230

3. When Does It Use Levenshtein?

Levenshtein helps compare text with small differences. See the Word Guide on page 1. The program turns the number of text changes into a similarity score.

In the **weighted duplicate checker**, it compares the available details even when the Student IDs are already the same. It is **not used only when an ID is missing or different**.

Account creation has a separate name check. After finding the same Student ID in the scholar list, it checks whether the names are close enough. A Student ID is required to create an account. A similar name cannot replace a missing Student ID.

The current process for creating or uploading scholar-list entries checks exact IDs and names after basic text cleaning. Older fuzzy checks are still written in that part of the program, but the process finishes or stops before reaching them. They do not run there. Fuzzy matching is still used when the admin checks for duplicate records and when a grantor edits a scholar's record.

4. What Scores Are Needed?

In the weighted checker, **any one** of these rules can mark two records as a duplicate:

Rule	What Must Happen
Same Student ID	An exact Student ID match is enough by itself.
Same email OR contact number	The email or contact number matches exactly, AND the names are at least 72% similar.
General similarity check	The names are at least 82% similar, AND the combined weighted score is at least 84%. Both are needed.

The admin's grouping check has an extra rule: it can group records when the combined score reaches its default 84%. For that rule, it does not always require the separate 82% name score.

The account-creation name check is different: it accepts a name score of at least 72%, using either the original or alphabetically arranged name words. It can also accept the same last name with at least 50% token overlap, which measures shared name words.

These numbers belong to different rules. Do not describe them as one score used everywhere. They are matching cutoffs, not the percentage of correct results.

Where this is written in the code: backend/grantor_algorithms.py:109, :206; backend/signup_service.py:519

5. What Happens to Each Scholarship Status?

A student's application status and scholarship commitment both matter. A status word by itself does not always decide whether the student can apply again.

Existing Status or Situation	What the Current Program Does
Active scholarship with a commitment or award	The commitment checks stop another conflicting scholarship commitment or application.
Pending application, with no commitment yet	The student may apply to different grantors. Another open application under the same grantor is restricted.
The same application is sent again	The system returns the saved application instead of making another copy.
Finished or completed requirements	This does not always end the scholarship. Finished may only mean the student completed the semester's requirements.
Cancelled/canceled, withdrawn, rejected, declined, denied, archived, resolved or expired	The shared application-status check treats these as closed. However, the system must still check any remaining commitment and other restrictions before allowing a new application.
Rejected, denied or declined	A 24-hour waiting period applies before applying again to the same grantor.
Inactive or terminated	Different parts of the program do not handle these words in the same way. Do not say that either word automatically allows a new application.

One more rule: if a grantor archives a student's scholar record, the student may need an invitation before applying to that grantor again.

Remember: 'I finished my documents' does not mean 'my scholarship has ended.' The program uses Finished for completed semester requirements. That alone does not remove a scholarship commitment.

Where this is written in the code: supabase/migrations/20260906124238_multiple_applications_one_choice.sql:130; 20260918130000_automatic_roster_and_login_security.sql:299

6. Does a Possible Duplicate Always Stop Approval?

No. It depends on what the user is doing.

- **Admin checks for similar records:** the system groups possible duplicates and hides extra entries in the Student List. This check alone does not stop account approval and does not delete the saved records.

- **Grantor edits a scholar's record:** if the fuzzy checker finds a duplicate, the screen prevents the grantor from saving the change.

- **Grantor uploads a scholar list:** identity or scholarship-assignment conflicts stop the import. The system also saves a conflict record for checking.

- **Student applies or requests materials:** an unresolved scholar-list conflict or an existing scholarship commitment can cause the server to stop the action.

- **Admin approves a Pending Account:** the account may still become active even if a scholar-list conflict exists. The system marks the conflict for staff to fix instead of assigning another scholarship.

For the paper: 'Every possible duplicate stops approval' is not correct. Making an account active and giving that student a scholarship are separate actions.

Where this is written in the code: supabase/migrations/20261010154625_allow_account_approval_with_roster_resolution.sql:90

Suggested Thesis Explanation in Simple English

The system uses exact identity checks, text-similarity checks and scholarship commitment rules. The weighted checker compares Student ID, full name, email, contact number, course, year level, city and province. Its general similarity rule requires a combined score of at least 84% and a name score of at least 82%. It also has separate rules for exact matching details. Possible duplicate records help staff review student information. Conflicting scholar-list assignments and existing commitments restrict scholarship actions. Approving an account is different from assigning a scholarship. Finishing semester requirements does not automatically end an active scholarship.